

錯誤與例外處理

- **24-1:語法錯誤 / 執行時期錯誤 / 邏輯錯誤**

語法錯誤.....	24-1
執行時期錯誤.....	24-2
邏輯錯誤.....	24-3

- **24-2:使用除錯工具**

中斷模式(Break Mode).....	24-4
使用Debug工具列.....	24-7
追蹤程式的執行.....	24-8
Debugging Windows.....	24-9

- **24-3:認識例外處理**

例外處理.....	24-10
Exception類別.....	24-12
常碰到的Exception類別.....	24-13
Exception類別常用成員.....	24-14

- **24-4:使用try/catch區段 / 多個catch區段**

使用try/catch區段.....	24-15
使用多個catch區段.....	24-18
自訂例外處理-throw敘述.....	24-19

Module 24

語法錯誤

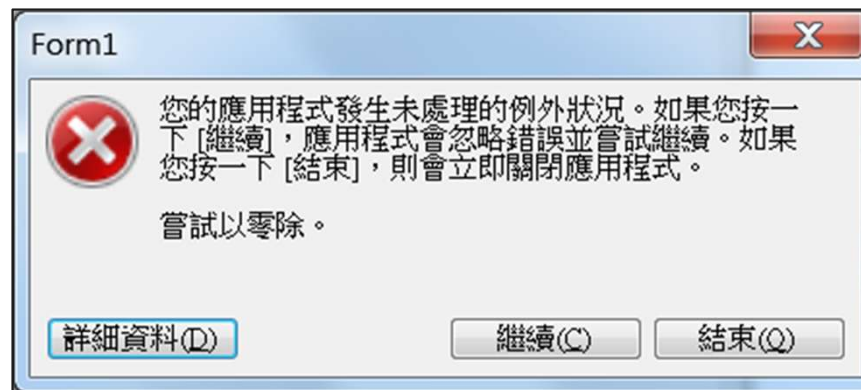
- 錯誤可分為三種，分別是語法錯誤、執行時期錯誤、邏輯錯誤。
- 語法錯誤(Syntax Error)，為編譯時期的錯誤，C#為嚴謹的語言，Visual Studio會即時檢查程式語法，若語法錯誤則無法編譯。



執行時期錯誤

- 執行時期錯誤(Runtime Exception)，編譯時語法沒有錯誤，執行時發生無法處理的例外狀況。

```
int hours = 0;  
int miles = 5;  
int speed = miles / hours;
```



邏輯錯誤

- 編譯及執行時都未發生錯誤，但執行結果卻不如預期或出錯。

```
string[] strArr = { "醒醒吧幻想肥宅", "知恩我婆", "IU" };  
  
for (int i = 1; i < strArr.Length; i++)  
{  
    Console.WriteLine(strArr[i]);  
}
```

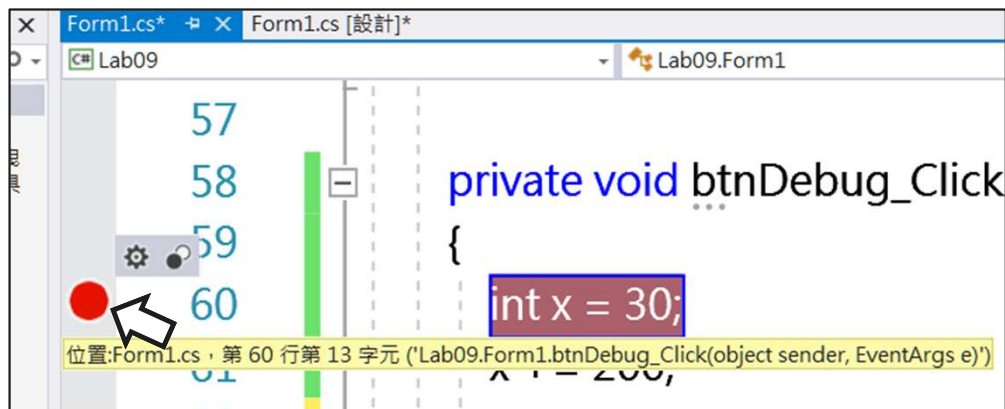


中斷模式(Break Mode)-1

- 為了除錯(Debug)而以中斷點中斷程式，可以在中斷模式中：
 - 追蹤應用程式的執行。
 - 檢視程序呼叫的順序。
 - 檢視變數，屬性的內容和運算式的結果。
 - 修改變數，屬性的內容。
 - 改變程式的流程。
 - 呼叫其他的程序/函數。

中斷模式(Break Mode)-2

- 設置中斷點：

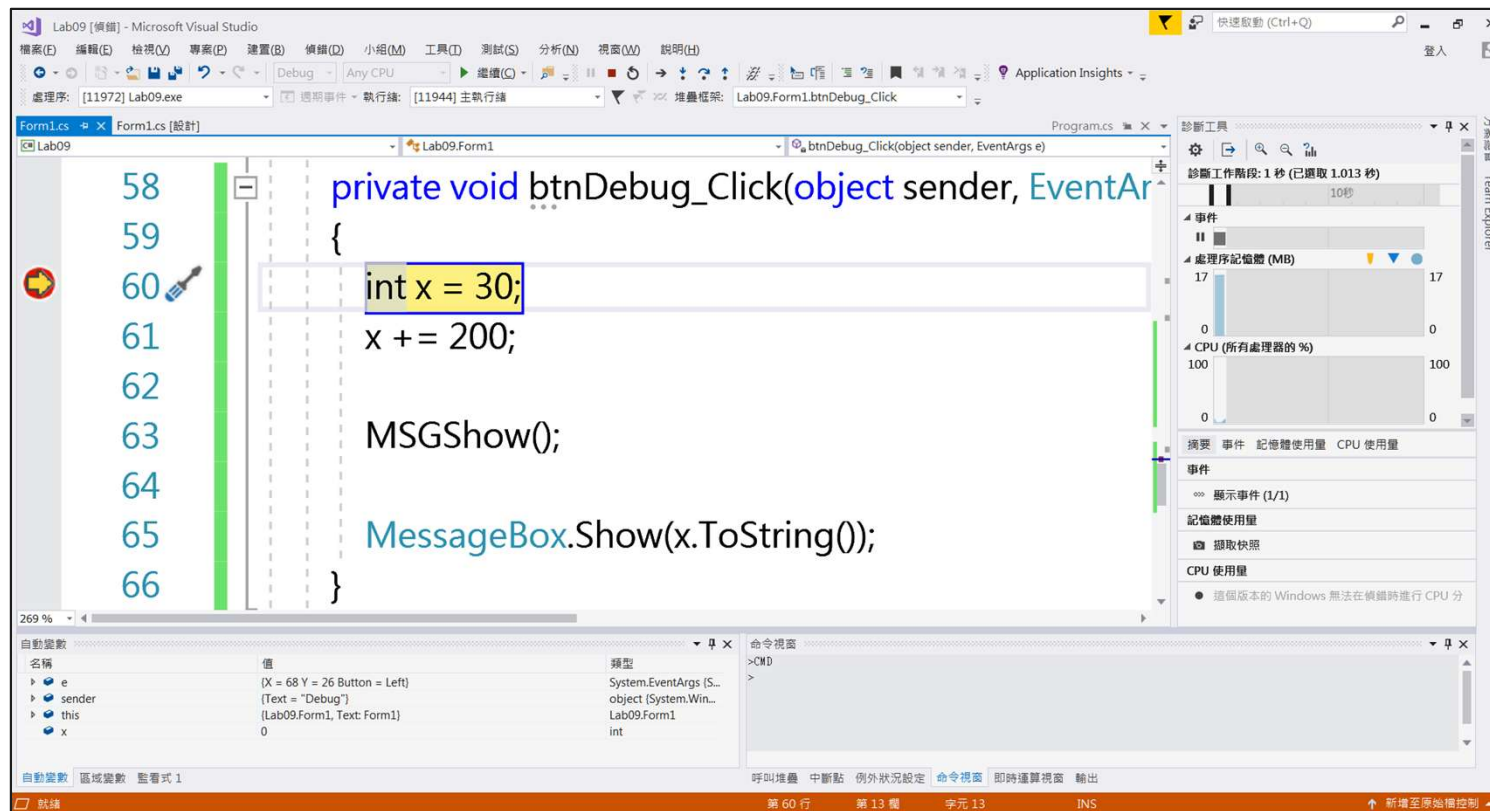


- 修改中斷點屬性：

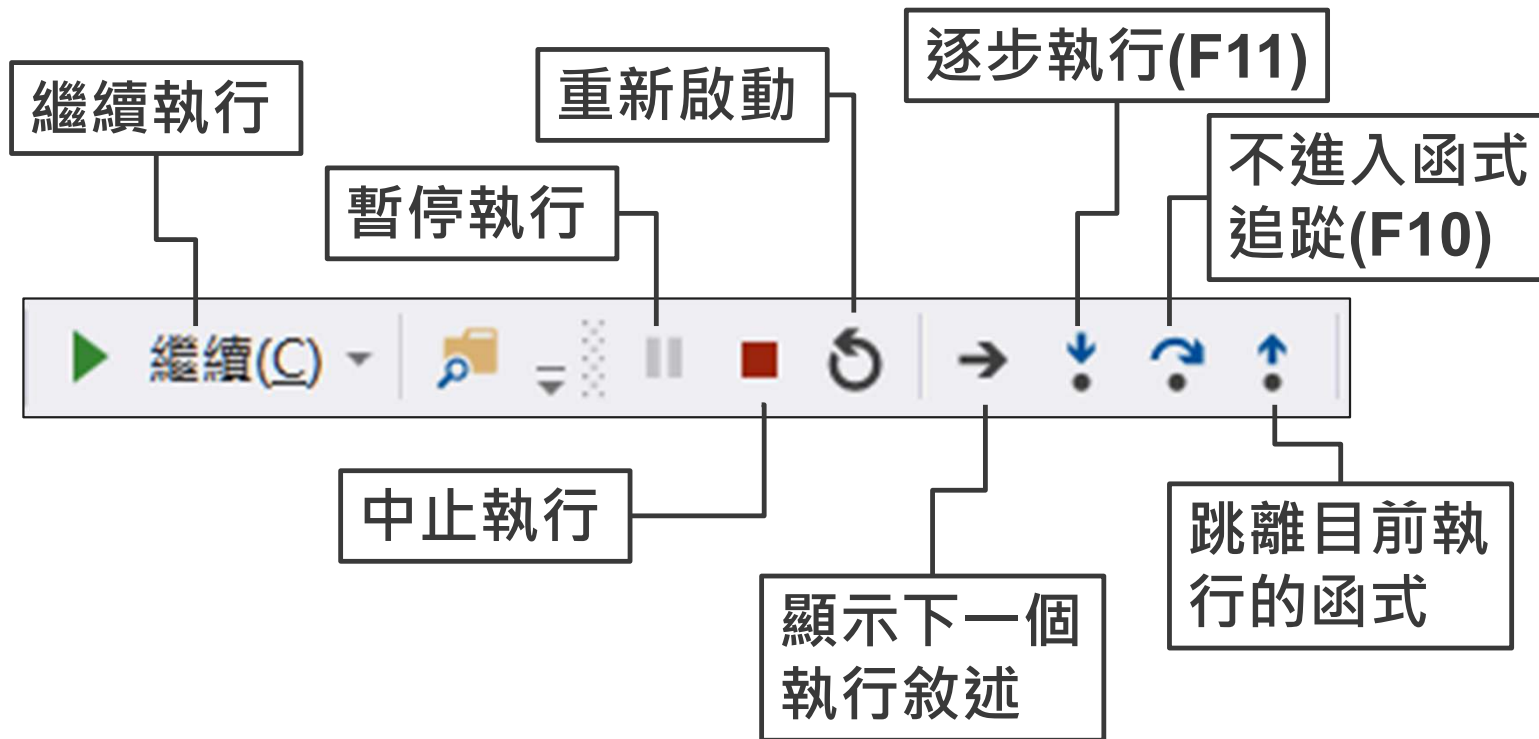


中斷模式(Break Mode)-3

- 偵錯執行：
 - 點下  開始 以進入中斷模式偵錯。



使用Debug工具列



追蹤程式的執行

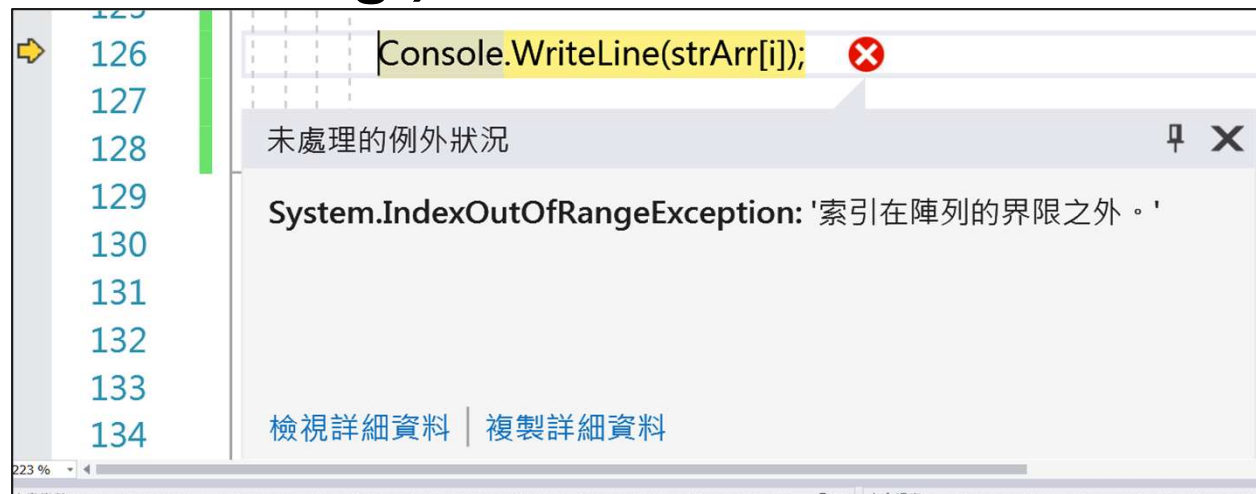
- **Step Into(逐步執行)：**
 - 逐步進入函數追蹤執行。
- **Step Over(不進入函式)：**
 - 不進入函式追蹤執行。
- **Step Out(跳離函式)：**
 - 跳離目前執行的函式追蹤。
- **Run to Cursor：**
 - 執行至游標處。

Debugging Windows

視窗	用途
自動變數(Autos)	觀察參數
呼叫堆疊(Call Stack)	觀察被呼叫但尚未結束的程序和函數
區域變數(Local)	觀察區域變數
監看式(Watch)	自訂欲觀察的變數或物件 自訂watch expression

例外處理-1

- 例外(Exception)：程式執行時，若發生問題或有異常導致程式無法繼續執行，此時系統會主動產生一個錯誤訊號「例外」。
- 例外處理(Exception Handle)：在某些情況下，開發者不希望程式遇到例外就中斷執行，而是希望能攔截例外並採取對應的處理措施，這個偵測、捕捉與處理例外的過程，就稱為「例外處理 (Exception Handling) 」。



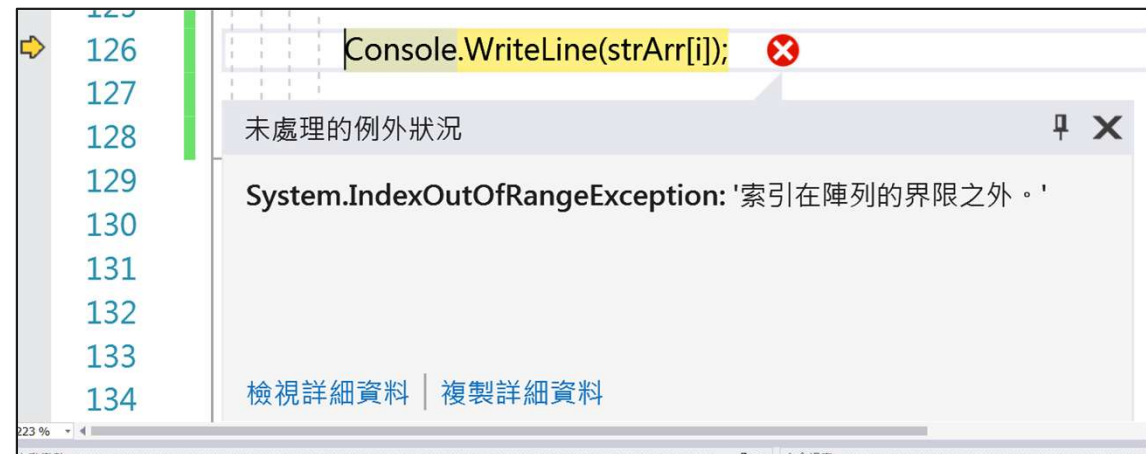
例外處理-2

- 早期還沒有「例外處理」時：

```
string[] strArr = { "醒醒吧幻想肥宅", "知恩我婆", "IU" };  
for (int i = 0; i < strArr.Length; i++)  
{  
    if (i >= strArr.Length)  
    {  
        goto Failed;  
    }  
    Console.WriteLine(strArr[i]);  
Failed:  
    Console.WriteLine("索引超出範圍");  
}
```

Exception 類別

- 在 .NET 中，例外（Exception）是一種物件，用來描述程式執行期間所發生的錯誤狀況。
- 每種錯誤類型都有其對應的 Exception 類別，這些類別都繼承自 System.Exception 基底類別。
- 如下圖，「索引在陣列的界限之外」的狀況以 System.IndexOutOfRangeException 表示。
- 透過例外類別，開發者可以針對不同錯誤執行對應處理，讓程式更具彈性與穩定性。



常碰到的Exception類別

例外類別	說明
ArgumentOutOfRangeException	當引數值超出呼叫方法所規定的範圍時
DivideByZeroException	除數為0時所產生的錯誤
Exception	程式執行時期所產生錯誤 其可補捉所有的例外
IndexOutOfRangeException	索引值超出陣列所允許的範圍
InvalidCastException	資料型別轉換所產生錯誤，如將字母轉為數值
OverflowException	溢位時所產生的錯誤

Exception類別常用成員

例外屬性	說明
HelpLink	用來讀取或設定說明檔的路徑
InnerException	用來取得造成目前例外的例外物件
Message	取得例外的描述訊息
Source	取得發生例外的來源物件
StackTrace	取得發生例外的函式
TargetSite	取得擲回(throw)目前例外狀況的方法
例外方法	說明
GetBaseException()	取得該例外類別所繼承的父義類別
GetObjectData()	用於子類別中覆寫義類別的方法，設定例外狀況的相關資訊
GetType()	取得目前例外物件的型別
ToString()	取得目前例外狀況的文字說明

使用try/catch區段-1

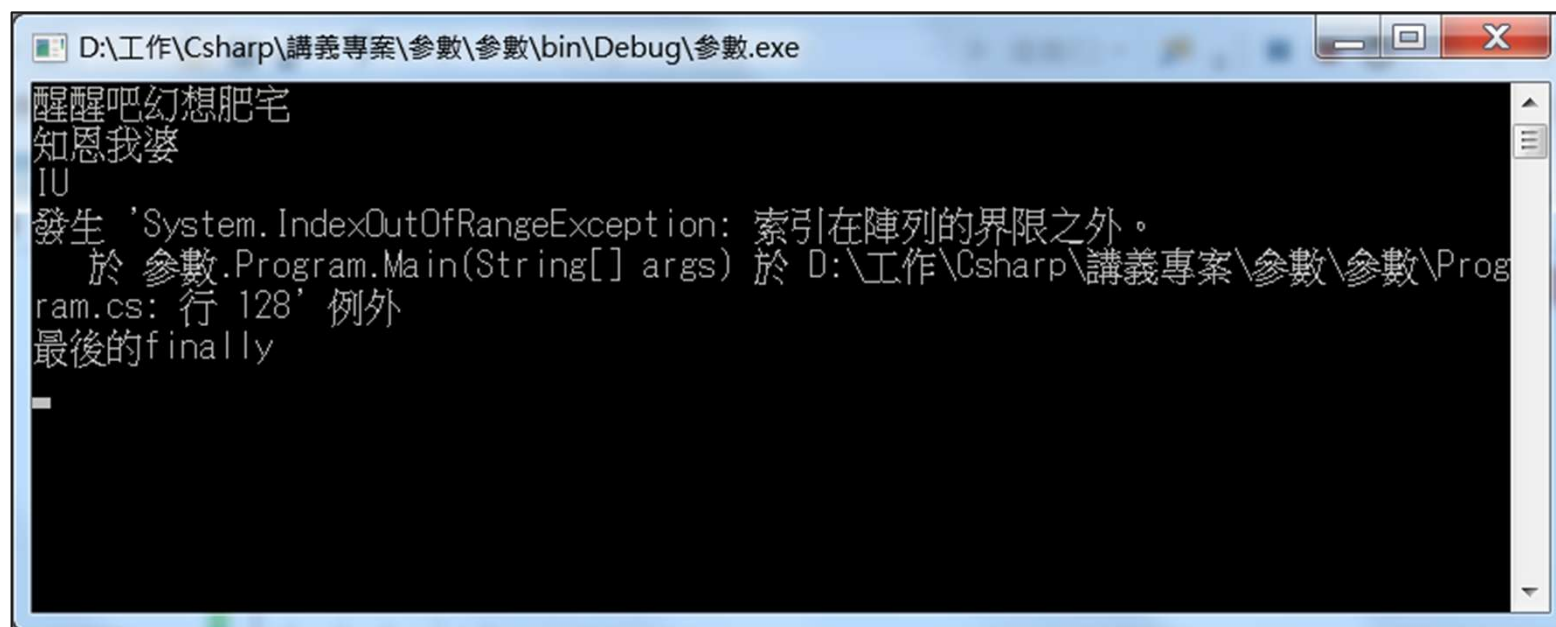
- **try { }** 中用來包覆特定想保護的程式碼，避免例外錯誤時導致整個應用程式癱瘓，若有例外狀況，則會跳至相應的**catch**。
- **catch ()** 中表達的是例外類別，並將在遭遇此例外時執行 { } 內的敘述。
- **finally { }** 中表達的是無論有沒有例外狀況都會執行的敘述，依需求而要設置。

使用try/catch區段-2

- 語法：
 - try
 - {
 - //...受保護的程式碼...
 - }
 - catch (例外類別)
 - {
 - //...發生例外類別要執行的敘述...
 - }
 - finally
 - {
 - //...最後都會執行的敘述...
 - }

```
string[] strArr = { "醒醒吧幻想肥宅", "知恩我婆", "IU" };  
try  
{  
    for (int i = 0; i < strArr.Length; i++)  
    {  
        Console.WriteLine(strArr[i]);  
    }  
} catch (IndexOutOfRangeException ex)  
{  
    Console.WriteLine($"發生 '{ex}' 例外");  
}  
finally  
{  
    Console.WriteLine("最後的finally");  
}
```

使用try/catch區段-3



```
D:\工作\Csharp\講義專案\參數\參數\bin\Debug\參數.exe
醒醒吧幻想肥宅
知恩我婆
IU
發生 'System.IndexOutOfRangeException: 索引在陣列的界限之外。'
    於 參數.Program.Main(String[] args) 於 D:\工作\Csharp\講義專案\參數\參數\Program.cs: 行 128' 例外
最後的finally
```

使用多個 catch 區段

- 語法：
 - **try**
 - {
//...受保護的程式碼...
 - catch** (例外類別1)
 - {
//...發生例外類別1要執行的敘述...
 - catch** (例外類別2)
 - {
//...發生例外類別2要執行的敘述...
 - finally**
 - {
//...最後都會執行的敘述...

```
string[] strArr = { "醒醒吧幻想肥宅", "知恩我婆", "TU" };  
try  
{  
    for (int i = 0; i < strArr.Length; i++)  
    {  
        Console.WriteLine(strArr[i]);  
    }  
}  
catch (IndexOutOfRangeException ex)  
{  
    Console.WriteLine($"發生 '{ex}' 例外");  
}  
catch (Exception ex)  
{  
    Console.WriteLine($"發生 '{ex}' 例外");  
}  
finally  
{  
    Console.WriteLine("最後的finally");  
}
```

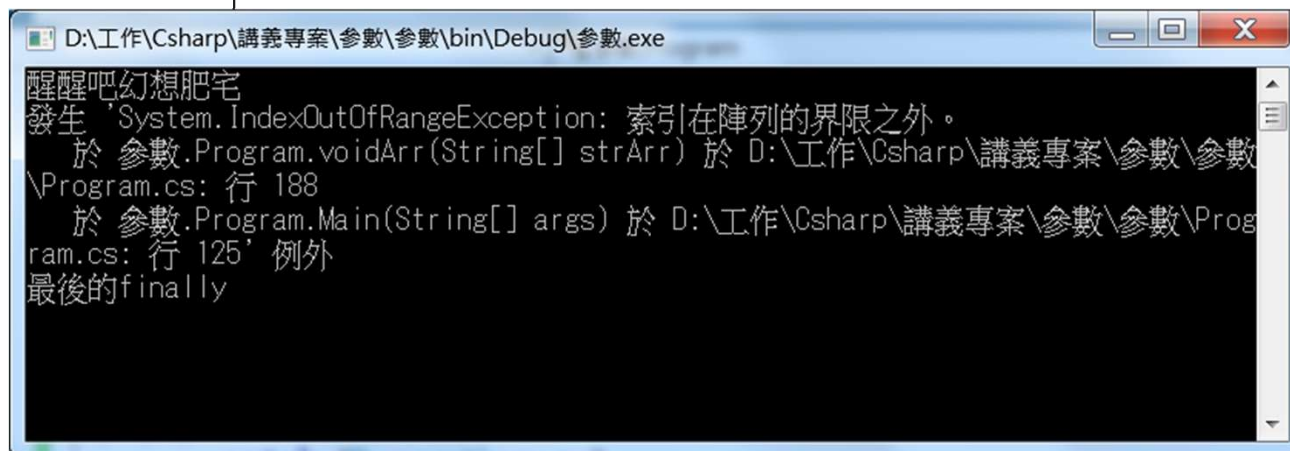
自訂例外處理-throw敘述-1

- **throw**：當程式遇到特殊錯誤狀況，而系統中沒有對應的內建例外類別時，開發者可以使用 **throw** 敘述主動拋出例外（**Exception**）。
- 可用於：
 - 拋出現有的例外類別（如 **ArgumentException**）
 - 拋出自訂的例外類別（繼承自 **System.Exception**）

```
static void voidArr ( string [] strArr)
{
    for (int i = 0; i < strArr.Length; i++)
    {
        if(strArr[i] == "知恩我婆")
        {
            throw new IndexOutOfRangeException();
        }
        Console.WriteLine(strArr[i]);
    }
}
```

自訂例外處理-throw敘述-2

```
string[] strArr = { "醒醒吧幻想肥宅", "知恩我婆", "IU" };  
try  
{  
    voidArr(strArr);  
}  
catch (IndexOutOfRangeException ex)  
{  
    Console.WriteLine($"發生 '{ex}' 例外");  
}  
catch (Exception ex)  
{  
    Console.WriteLine($"發生 '{ex}' 例外");  
}  
finally  
{  
    Console.WriteLine("最後的finally");  
}
```



```
D:\工作\Csharp\講義專案\參數\參數\bin\Debug\參數.exe  
醒醒吧幻想肥宅  
發生 'System.IndexOutOfRangeException: 索引在陣列的界限之外。  
    於 參數.Program.voidArr(String[] strArr) 於 D:\工作\Csharp\講義專案\參數\參數  
    \Program.cs: 行 188  
    於 參數.Program.Main(String[] args) 於 D:\工作\Csharp\講義專案\參數\參數\Prog  
    ram.cs: 行 125' 例外  
最後的finally
```